

# Guide to the $k$ -NN Colab Exercises

Companion to `Exercises_KNN.ipynb`

**How to use this guide.** Read the *key material* section before starting. Then, for each question, try on your own first. If you are stuck for more than a few minutes, open the hints in order: **Hint 1** points you in the right direction, **Hint 2** gives the method, **Hint 3** takes you to the last step. The hints never give the final numbers: the point is that you produce them. The *common mistakes* boxes list what goes wrong most often; read them even when you think you are done.

## Key material

### The $k$ -NN classifier in one paragraph

$k$ -NN stores the training set. To classify a query  $q$ : compute the distance from  $q$  to every training point, keep the  $k$  nearest, and return the most frequent class among them. There is no training in the usual sense; all the work happens at prediction time. The only choices are  $k$ , the distance, the weighting of votes, and **the scale of the features**, since the distance mixes all features together.

### NumPy broadcasting: the tool for Exercise 1

Two arrays can be combined element-wise when, comparing shapes from the right, each pair of dimensions is equal or one of them is 1. A dimension of size 1 is “stretched”.

```
a = np.array([1, 2, 3])           # shape (3,)
b = np.array([10, 20])           # shape (2,)
a[:, None] + b[None, :]          # shapes (3,1) and (1,2) -> (3,2)
# array([[11, 21],
#        [12, 22],
#        [13, 23]])
```

`None` (or `np.newaxis`) inserts a new axis of size 1. With 2-D arrays  $A$  of shape  $(n_A, p)$  and  $B$  of shape  $(n_B, p)$ , think about which axes to insert so that every row of  $A$  meets every row of  $B$ , and along which axis you then sum.

### Useful NumPy functions

|                                                         |                                                                                     |
|---------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>np.argsort(M, axis=1)</code>                      | for each row, the column indices that would sort that row                           |
| <code>M[:, :k]</code>                                   | the first $k$ columns                                                               |
| <code>y[idx]</code> ( <code>idx</code> a 2-D int array) | “fancy indexing”: an array of the same shape as <code>idx</code> holding the labels |
| <code>np.bincount(v, minlength=c)</code>                | counts of $0, 1, \dots, c-1$ in the integer vector $v$                              |
| <code>counts.argmax(axis=1)</code>                      | index of the largest value per row (the first one in case of a tie)                 |

**The model-selection protocol used in every exercise**

1. Split once into training and test sets (`train_test_split`, `stratify=y` for classification).
2. Put preprocessing and model in a **pipeline**: `make_pipeline(StandardScaler(), KNeighborsClassifier())`.
3. Choose hyperparameters by cross-validation on the training set only (`cross_val_score`, `GridSearchCV`).
4. Refit the chosen model on the whole training set and evaluate it **once** on the test set.

Inside a pipeline, parameters are addressed as `<step name>__<parameter>`, where the step name is the lowercase class name, e.g. `standardscaler`, `kneighborsclassifier`.

**Mathematical insight · what  $k$ -NN is estimating**

The best possible classifier (the *Bayes classifier*) predicts the class with the largest conditional probability  $P(Y = c \mid X = x)$ .  $k$ -NN estimates that probability by the fraction of neighbours in class  $c$ :

$$\hat{P}(Y = c \mid X = q) = \frac{1}{k} \sum_{i \in N_k(q)} \mathbb{1}[y_i = c].$$

This is a *local average*. Two things must happen for it to be accurate: the neighbours must be close to  $q$  (so that their class probabilities resemble those at  $q$ ), which needs  $k$  small relative to  $n$ ; and there must be many of them (so that the average is not noisy), which needs  $k$  large. Formally,  $k$ -NN is consistent when  $k \rightarrow \infty$  and  $k/n \rightarrow 0$ . Every choice of  $k$  is a compromise between these two requirements: this is the bias–variance trade-off.

A classical result (Cover and Hart, 1967) says that, with infinitely many data, the error of 1-NN is at most twice the Bayes error. So even the crudest version is not a bad classifier, but its variance on finite data is large.

**Mathematical insight · bias and variance, made explicit**

For  $k$ -NN *regression* with  $y = f(x) + \varepsilon$ ,  $\text{Var}(\varepsilon) = \sigma^2$ , and fixed training inputs, the expected squared error at  $q$  decomposes as

$$\mathbb{E}[(y - \hat{f}(q))^2] = \sigma^2 + \left(f(q) - \frac{1}{k} \sum_{i \in N_k(q)} f(x_i)\right)^2 + \frac{\sigma^2}{k}.$$

The last term (variance) falls like  $1/k$ ; the middle term (bias) grows as the neighbourhood widens and includes points where  $f$  is different. A useful measure of complexity is the *effective number of parameters*  $\approx n/k$ : 1-NN has about  $n$  “parameters”,  $k = n$  has one (the global mean).

**Mathematical insight · cost of prediction**

Brute force needs  $O(np)$  operations per query (one distance to each of the  $n$  training points in dimension  $p$ ) plus the selection of the  $k$  smallest. Tree structures (KD-trees, ball trees; `parameter algorithm` in `scikit-learn`) avoid most distance computations in low dimension, but lose their advantage when  $p$  is large, for the reasons of Exercise 3.

**Exercise 1  $k$ -NN from scratch**

## 1.2 pairwise\_sq\_dists(A, B)

**Hint 1 · Nudge** The output has one row per point of A and one column per point of B. Start by writing down the shape of every intermediate array.

**Hint 2 · Method** The hint in the notebook creates an array of shape  $(n_A, n_B, p)$  holding all the differences. Square it element-wise and reduce over the *feature* axis.

**Hint 3 · Almost there** The feature axis is the last one: `axis=2` (or `axis=-1`). Test on the toy example of the CHECK cell by hand before running it: the distance from  $(0,0)$  to  $(3,4)$  must be 25.

### Going further (optional)

The array of differences has  $n_A \times n_B \times p$  entries and can be large. The identity  $\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2a \cdot b$  lets you compute the same matrix with one matrix product,  $A @ B.T$ , plus two vectors of squared norms. That is what scikit-learn does. Rounding errors can make tiny values slightly negative; `np.maximum(D, 0)` fixes that.

## 1.3 knn\_predict

**Hint 1 · Nudge** Follow the three steps written in the notebook, one line each.

**Hint 2 · Method** Step 1 reuses your function (queries first, training points second). Step 2: `np.argsort` on each row, keep the first  $k$  columns. Step 3: turn indices into labels with `y_train[idx]`, then count.

**Hint 3 · Almost there** `np.bincount` works on one 1-D vector at a time. You can loop over the rows (a list comprehension is fine here), or use `np.apply_along_axis`. The number of classes is `y_train.max() + 1`; pass it as `minlength` so that every row has the same length.

### Common mistakes

- Calling `pairwise_sq_dists(X_train, X_query)`: the shape is transposed and `argsort(axis=1)` then sorts the wrong thing.
- Sorting the distances (`np.sort`) instead of getting indices (`np.argsort`): you need *which* points are close, not how close.
- Forgetting `minlength`: a row where class 2 gets no vote produces a shorter count vector.

### Why the CHECK uses jittered data

The iris measurements are rounded to 0.1 cm, so several training points are often at exactly the same distance from a query. Your code and scikit-learn may break such ties differently and legitimately disagree. Adding noise of size  $10^{-4}$  removes all ties without changing the problem, so on that copy the two implementations must agree exactly.

### Mathematical insight · the shape of the decision boundary

For 1-NN, the plane is divided into *Voronoi cells*: the cell of  $x_i$  is the set of points closer to  $x_i$  than to any other training point. The border between two cells is a piece of the perpendicular bisector of the two points (the set where  $\|q - x_i\| = \|q - x_j\|$ , which is linear in  $q$  after ex-

panding the squares). The 1-NN decision boundary is therefore a union of straight segments following the training points exactly. Increasing  $k$  merges and smooths these pieces.

## 1.4 Decision regions

**Hint 1 · Nudge** `plot_regions` needs a *function* that takes a grid of points and returns their classes.

**Hint 2 · Method** Build it with a lambda: `lambda G: knn_predict(X_train, y_train, G, k)`.

**Hint 3 · Almost there** Inside a loop, a lambda captures the variable  $k$ , not its current value. Freeze it with a default argument: `lambda G, k=k: ....`

## 1.5 Accuracy versus $k$

**Hint 2 · Method** Loop over  $k$ s, fit `KNeighborsClassifier(n_neighbors=k)` on the training set and call `.score` twice: once on the training data, once on the test data.

### Question 1 (written)

**Hint 1 · Nudge** (a) Look for small isolated islands of colour and for straight, blocky borders. Relate each to overfitting or underfitting, and quote the train and test accuracies from 1.5 for the same  $k$ .

**Hint 2 · Method** (b) With  $k = 1$  a training point's nearest neighbour is normally itself. When can the nearest point be a *different* point? The cell prints the number of duplicated training points: think about duplicates with different species.

**Hint 3 · Almost there** (c) If you pick the best of 60 test scores, is that score still an unbiased estimate of performance on new data? What remains to evaluate the chosen model?

## Exercise 2 Breast cancer diagnosis

### Read the label coding carefully

In this dataset **0 = malignant** and **1 = benign**. Many scikit-learn metrics treat class 1 as the “positive” class by default. To measure how well malignant tumours are detected, pass `pos_label=0` to `recall_score` and `precision_score`.

- **Recall (malignant):** of all malignant tumours, the fraction the model flags. A missed cancer lowers it.
- **Precision (malignant):** of all tumours flagged malignant, the fraction that really are. A false alarm lowers it.

### Mathematical insight · standardising changes the metric

Standardising and then using the Euclidean distance is the same as using, on the raw data,

the weighted distance

$$d(u, v) = \sqrt{\sum_j ((u_j - v_j)/s_j)^2} = \sqrt{(u - v)^\top D^{-1}(u - v)}, \quad D = \text{diag}(s_1^2, \dots, s_p^2).$$

So the choice of scaling is the choice of a metric. Replacing  $D$  by the full covariance matrix gives the Mahalanobis distance, which also accounts for correlations between features; metric-learning methods go further and learn the matrix from the labels.

## 2.2 Does scaling matter?

**Hint 2 · Method** `cross_val_score(model, X, y, cv=cv)` returns one accuracy per fold; take `.mean()`. Call it once with the bare classifier and once with the pipeline.

**Hint 3 · Almost there** Pass the *training* split (`Xb_train, yb_train`), never the full data.

### Question 2 (written)

**Hint 1 · Nudge** In the table of 2.1, compare the smallest and the largest standard deviations. By how many orders of magnitude do they differ?

**Hint 2 · Method** A feature with a spread in the hundreds contributes squared differences in the tens of thousands. What is left for a feature whose spread is 0.002?

**Hint 3 · Almost there** For the leakage part: if the scaler is fitted on all the data, which statistics of the validation fold have entered the training process?

## 2.3 Grid search

**Hint 1 · Nudge** The keys of `param_grid` use the `step__parameter` convention; the example key is given in the comment.

**Hint 2 · Method** `list(range(1, 42, 2))` gives the odd numbers from 1 to 41. After fitting, look at `search.best_params_`, `search.best_score_` and `search.cv_results_`.

**Hint 3 · Almost there** `pd.DataFrame(search.cv_results_)` has columns `mean_test_score`, `std_test_score` and `param_...` for each hyperparameter. Group by the `weights` column to draw one curve per scheme, and use `plt.errorbar(x, y, yerr=...)`.

### Standard error and the one-standard-error rule

The CV score is an average of 5 fold scores, each noisy. Its standard error is approximately  $SE = \text{std}/\sqrt{5}$ . Two models whose mean scores differ by less than about one SE cannot be told apart by these data. The one-SE rule: among all models within one SE of the best mean score, choose the **simplest**.

### Mathematical insight · where the standard error comes from, and its limits

If the 5 fold scores  $s_1, \dots, s_5$  were independent with variance  $\tau^2$ , their mean would have variance  $\tau^2/5$ , estimated by  $\text{std}^2/5$ . In reality the folds share most of their training data,

so the scores are positively correlated and  $\text{std}/\sqrt{5}$  *underestimates* the true uncertainty. The one-SE rule is therefore a conservative heuristic, not a statistical test: if anything, the real noise is larger, which makes preferring the simpler model even more reasonable.

## 2.4 The one-SE choice

**Hint 1 · Nudge** What makes a  $k$ -NN model simpler: a small  $k$  or a large  $k$ ? Think of  $k = 1$  versus  $k = n$ .

**Hint 2 · Method** Keep only the `uniform` rows. Find the row with the highest mean score; the threshold is its mean minus its SE. Among the rows at or above the threshold, which  $k$  do you pick?

**Hint 3 · Almost there** Remember to convert the  $k$  column to integers before comparisons (`.astype(int)`); values in `cv_results_` are stored as objects.

### Question 3 (written)

**Hint 2 · Method** Measure the gap between the best mean score and the score of your one-SE choice, and compare it to the SE. Then translate it into tumours: roughly how many validation tumours are in one fold?

## 2.5 Test set, once

**Hint 2 · Method** Fit the pipeline with `n_neighbors=k_lse` on `Xb_train`, then `.score(Xb_test, yb_test)`. `DummyClassifier()` predicts the most frequent class; its accuracy is your “doing nothing” baseline. `ConfusionMatrixDisplay.from_estimator(model, X, y)` draws the matrix.

### Mathematical insight · the optimal threshold from costs

Suppose a missed malignant tumour costs  $C_{FN}$  and a false alarm costs  $C_{FP}$ . For a tumour with malignant probability  $p$ , flagging it has expected cost  $(1 - p)C_{FP}$  and not flagging it has expected cost  $pC_{FN}$ . Flag it when  $pC_{FN} > (1 - p)C_{FP}$ , i.e. when

$$p > t^* = \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

Equal costs give  $t^* = 0.5$ ; a miss that is 4 times worse than a false alarm gives  $t^* = 0.2$ . This assumes the probabilities are well calibrated, which  $k$ -NN vote fractions only roughly are (they take only the values  $0, 1/k, \dots, 1$ ).

## 2.6 Moving the threshold

**Hint 1 · Nudge** `predict` uses a threshold of 0.5 on the vote fraction. With `predict_proba` you get the vote fractions and can choose any threshold.

**Hint 2 · Method** `cross_val_predict(model, X, y, cv=cv, method="predict_proba")` returns, for each training tumour, probabilities computed by a model that did *not* see it. Column 0 is the malignant probability.

**Hint 3 · Almost there** For a threshold  $t$ : `pred = np.where(p_malignant >= t, 0, 1)`. Collect one dictionary per threshold in a list, then `pd.DataFrame(rows)`.

### Question 4 (written)

**Hint 1 · Nudge** Read your table: what do you gain and what do you lose when  $t$  goes down?

**Hint 2 · Method** Weigh the two errors in a screening context: what does a false alarm cost, and what does a missed cancer cost?

## Exercise 3 The curse of dimensionality

### Distance concentration

For points spread uniformly in high dimension, the distance from a query to its nearest and to its farthest neighbour become almost equal: the ratio  $(d_{\max} - d_{\min})/d_{\min}$  goes to 0. “Near-est” then carries little information. Each irrelevant feature adds its own random contribution to every distance, which has the same effect on real data.

### Mathematical insight · why distances concentrate

Let  $X, Y$  be independent and uniform in  $[0, 1]^d$ . Each coordinate contributes  $Z_j = (X_j - Y_j)^2$ , with  $\mathbb{E}[Z_j] = 1/6$  and a fixed variance  $v$ . The squared distance is a sum of  $d$  independent terms, so

$$\mathbb{E}\|X - Y\|^2 = \frac{d}{6}, \quad \text{sd}(\|X - Y\|^2) = \sqrt{dv}.$$

The typical spread relative to the typical size is  $\sqrt{dv}/(d/6) \propto 1/\sqrt{d}$ , which goes to 0. All distances from a query crowd around the same value  $\sqrt{d/6}$ , so the nearest and farthest points are almost equally far. Real data escape this only partly, because their features are correlated and they lie near a lower-dimensional structure; noise features destroy exactly that structure.

### 3.1 distance\_contrast

**Hint 2 · Method** `rng.random((n, d))` draws an  $n \times d$  array uniform in  $[0, 1]$ ; `rng.random(d)` draws the query. Compute the  $n$  Euclidean distances with broadcasting (no need for your 2-D function), then the ratio.

**Hint 3 · Almost there** Use a log-log plot: `plt.loglog(dims, contrasts, "o-")`.

### 3.2 Adding noise features

**Hint 1 · Nudge** The noise columns are appended to the *training* features only; the test set is not used in this exercise.

**Hint 2 · Method** `np.hstack([Xtr, rng.normal(size=(len(Xtr), m))])` adds  $m$  columns (for  $m = 0$  it adds none). Then run `cross_val_score` with each of the two pipelines.

**Hint 3 · Almost there** Create `rng` once, before the loop, so the results match the solution.

### Question 5 (written)

**Hint 1 · Nudge** Quote the contrast for  $d = 1$  and  $d = 1000$ , and the two accuracy curves at  $m = 0$  and  $m = 1000$ .

**Hint 2 · Method**  $k$ -NN gives every feature the same weight in the distance. What does logistic regression do with a feature that does not help predict the label?

**Hint 3 · Almost there** Remedies: think of methods that remove features, that compress them, or that learn how much each feature should count.

#### Check yourself before moving on

- Can you explain, without code, why  $k$ -NN needs scaled features?
- Can you say why the test set was used only once in the whole notebook?
- Can you explain why the “simplest”  $k$ -NN model has the *largest*  $k$ ?
- Can you describe what `predict_proba` returns for  $k$ -NN?